

02

 UNIDAD I · FUNDAMENTOS DE SISTEMAS DISTRIBUIDOS CONFIABLES

INTEGRIDAD VERIFICABLE: HASH Y ÁRBOLES DE MERKLE

Blockchain y Web 3.0 · Ingeniería de Sistemas
Universidad de San Buenaventura Medellín

180 MIN

DURACIÓN

DEMO +
LABORATORIO

MODALIDAD

PYTHON 3
O NODE.JS

EQUIPO

REPOSITORIO

ENTREGABLE



LO QUE HAY QUE TRAER

01

PYTHON 3 O NODE.JS

Instalado y funcionando desde la línea de comandos. Se anunció la sesión pasada. El bloque D empieza en el minuto 120 y no hay tiempo para instalaciones.

02

EL WHITEPAPER LEÍDO

Secciones 1 a 5. Abrimos con la plenaria de la pregunta de la semana.

03

PAPEL Y LAPICERO

Para la frase semilla del bloque B. No sirve el teléfono, y la razón se explica ahí mismo.



SI LLEGA SIN ENTORNO

Trabajen en pareja con alguien que sí lo tenga, y resuelvan su instalación fuera de clase.



MAPA DE LA SESIÓN

MIN	BLOQUE	CONTENIDO
00–10	Apertura	Plenaria de la pregunta de la semana: ¿qué garantiza una cadena sobre la veracidad de lo escrito?
10–55	BLOQUE A	Demostración en vivo. Siete demostraciones proyectadas: de la clave a la cadena.
55–70	BLOQUE B	Billetera propia. Cada estudiante replica lo que acaba de ver.
70–80	C.1	Qué es un hash y qué no es. Las cuatro confusiones que salen caras.
80–92	C.2	Las propiedades exigibles y qué se rompe si falla cada una.
92–100	C.3	SHA-256 y Keccak-256. La trampa de que Keccak-256 no es SHA3-256.
100–112	C.4	Árboles de Merkle: probar que algo está en un bloque sin descargar el bloque.
112–120	Pausa	—
120–170	BLOQUE D	Laboratorio 01: hash, efecto avalancha, árbol de Merkle y prueba de inclusión.
170–180	BLOQUE E	Cierre, entregable y puente hacia las firmas digitales.

RESULTADOS DE APRENDIZAJE

RA	RESULTADO DE APRENDIZAJE	NIVEL	DÓNDE SE EVIDENCIA
RA1	Explicar los fundamentos criptográficos que sustentan la integridad de un registro distribuido.	Comprender	Núcleo de esta sesión. Laboratorio + Quiz 1 (S4)
RA4	Construir una suite de pruebas automatizadas.	Aplicar	Se inicia aquí con las pruebas del laboratorio. Se evalúa en S8.

 LAS TRES AFIRMACIONES QUE HOY SE DEMUESTRAN

- 01 Un cambio mínimo altera la mitad de la salida.
- 02 Una raíz de Merkle compromete el conjunto entero.
- 03 La pertenencia se prueba con una fracción logarítmica de los datos.

A



MIN 10 – 55 · EL DOCENTE EJECUTA Y NARRA

DEMOSTRACIÓN EN VIVO

Siete demostraciones proyectadas. Convierte en datos concretos todo lo que la Sesión 1 dejó en abstracto.



QUÉ DEJA CADA DEMOSTRACIÓN

#	DEMOSTRACIÓN	LA IDEA QUE DEBE QUEDAR
01	Crear una identidad desde cero	Una cuenta válida en una red global en menos de un minuto, sin nombre, sin documento y sin que nadie la apruebe.
02	La frase semilla, en pantalla	Doce palabras reconstruyen la cuenta completa en otro navegador y sin contraseña. La frase no da acceso: ES la billetera.
03	Anatomía de una dirección	Cuarenta y dos caracteres, y las mayúsculas no son decorativas: codifican una suma de verificación.
04	Cambiar de red	La misma dirección en dos libros independientes con saldos distintos.
05	Firmar: la pantalla que nadie lee	Firmar es irreversible. La mayoría de los robos no rompe criptografía: consigue que la víctima firme.
06	El explorador, campo por campo	Todo lo de la sesión pasada aparece como datos concretos. Es la demostración central.
07	La cuenta de un desconocido	Seudónimo no es anónimo: sin nombre, pero todo enlazado y permanente.



EL GANCHO HACIA EL BLOQUE CONCEPTUAL

En la demostración 06 apareció un campo llamado «Transaction Hash», y se dijo que cambia por completo si se altera un solo byte. Eso es una afirmación fuerte.

En hora y media la habremos demostrado — no de palabra, sino midiéndola en código propio.

B



MIN 55 – 70 · CADA ESTUDIANTE EN SU MÁQUINA

BILLETERA PROPIA

Quince minutos, inmediatamente después de haberlo visto hacer. Es el momento de máxima comprensión.



TRES REGLAS QUE NO ADMITEN EXCEPCIÓN

Se enunciaron en la Sesión 1. A partir de este momento aplican de verdad, porque cada quien tiene su propia billetera.

01

Nunca se ingresa dinero real en la billetera del curso.

Quien quiera operar con fondos reales debe usar otra, creada fuera de clase, en un equipo de su propiedad.

02

La frase de recuperación no se comparte con nadie. Nunca. Tampoco con el docente.

Nadie legítimo la pedirá jamás. Cualquiera que la pida está intentando robar.

03

No se firma ninguna transacción cuyo contenido no se comprenda.

Firmar es irreversible y puede autorizar a un tercero a mover todos los fondos de la cuenta.



CINCO PASOS, QUINCE MINUTOS

01

INSTALAR DESDE LA TIENDA OFICIAL

Verificar desarrollador y número de instalaciones. Existen falsificaciones con ícono casi idéntico cuyo único fin es capturar la frase semilla.

02

CREAR Y ANOTAR LAS DOCE PALABRAS

En papel, a mano, en orden. No captura, no foto, no archivo, no nota del teléfono, no correo a sí mismo.

03

HABILITAR LA RED DE PRUEBA

Verificar en el selector antes de continuar. La contraseña del navegador no es la cuenta y no la recupera en otro equipo.

04

RECIBIR FONDOS DE PRUEBA

El docente distribuye desde la billetera institucional. No se depende de faucets públicos en clase.

05

ENVIAR LA DIRECCIÓN AL CANAL DEL CURSO

Solo la dirección, que es pública. Sirve para el punto de control de la Sesión 5 y para las distribuciones siguientes.



C



MIN 70 – 112 · EXPOSICIÓN DIALOGADA

INTEGRIDAD VERIFICABLE

Qué es realmente un hash, qué propiedades debe cumplir y cómo se resume un conjunto entero en un solo valor.



QUÉ ES UNA FUNCIÓN HASH, Y QUÉ NO ES

En la Sesión 1 usamos la palabra hash tres veces sin definirla con rigor: al describir el enlace entre bloques, al derivar una dirección desde una clave pública y al identificar una transacción. Aquí se salda esa deuda.

FICHA DE TÉRMINO 01 · FUNCIÓN HASH CRIPTOGRÁFICA

Función matemática que toma una entrada de LONGITUD ARBITRARIA y produce una salida de LONGITUD FIJA, de forma determinista, eficiente de calcular en un sentido e inviable de invertir.

La entrada puede ser un carácter o una película de tres horas: la salida siempre mide lo mismo. En SHA-256, 256 bits, que se escriben como 64 caracteres hexadecimales.

Nótese la asimetría, que es toda la utilidad: calcular el hash de un dato es instantáneo; recuperar el dato a partir del hash es inviable con cualquier cómputo concebible.



UN HASH NO ES CIFRADO, NI COMPRESIÓN, NI AZAR, NI FIRMA

Casi todo lo que sale mal con funciones hash en la práctica profesional viene de confundirlas con otra cosa.

UN HASH NO ES...	POR QUÉ
...cifrado	El cifrado es REVERSIBLE: existe una clave que devuelve el texto original, y ese es su propósito. Un hash no tiene clave y no tiene vuelta. «Hasheada» y «encriptada» no son lo mismo, y confundirlo cambia por completo el análisis de riesgo.
...compresión	La compresión conserva la información: por eso se puede descomprimir. El hash DESTRUYE información. Un archivo de 4 GB y uno de 4 bytes producen salidas del mismo tamaño.
...aleatorio	Es completamente DETERMINISTA: la misma entrada produce siempre la misma salida, en cualquier máquina y momento. Parece aleatorio, que es distinto. Esa apariencia es una propiedad buscada, no azar.
...una firma	Prueba que un dato no cambió; NO prueba quién lo escribió. Cualquiera puede calcular el hash de cualquier cosa. La autoría requiere criptografía asimétrica: Sesión 3.



LAS COLISIONES EXISTEN. NECESARIAMENTE.



Entradas posibles: cualquier archivo, de cualquier tamaño.

2^{256}

Salidas posibles en SHA-256. Un número finito.



Al meter infinitos elementos en casillas finitas, hay casillas con más de uno.

Las colisiones no solo son posibles: es matemáticamente seguro que existen. La seguridad no consiste en que no existan, sino en que NADIE SEA CAPAZ DE ENCONTRAR UNA.

El número 2^{256} es difícil de dimensionar. Una comparación que suele funcionar en clase: es del orden de la cantidad estimada de átomos en el universo observable. Encontrar dos entradas con el mismo hash de 256 bits por fuerza bruta no es difícil — es imposible con la física conocida.

NUNCA USES SHA-256 PARA GUARDAR CONTRASEÑAS

LA APLICACIÓN EQUIVOCADA MÁS FRECUENTE

SHA-256 está diseñada para ser RÁPIDA, y esa es exactamente la propiedad que no se quiere al almacenar contraseñas: un atacante con la base de datos filtrada puede probar miles de millones de candidatos por segundo.

Para contraseñas se usan funciones deliberadamente lentas y con sal —bcrypt, scrypt, Argon2—, diseñadas para que cada intento cueste. Es un dominio distinto con requisitos opuestos.

EN BLOCKCHAIN

Queremos velocidad: cada nodo verifica millones de hashes. SHA-256 y Keccak-256 son la elección correcta.

EN AUTENTICACIÓN

Queremos lentitud: que cada intento del atacante cueste. Son la elección incorrecta.



LAS PROPIEDADES EXIGIBLES

Que una función produzca salida de longitud fija no la hace criptográfica. Léase la última columna: dice qué se rompería en una blockchain si esa propiedad fallara.

PROPIEDAD	QUÉ EXIGE	QUÉ SE ROMPE SI FALLA
Determinismo	La misma entrada produce siempre la misma salida.	Todo. Dos nodos calcularían hashes distintos para el mismo bloque y la red nunca alcanzaría acuerdo.
Eficiencia	Calcular el hash debe ser rápido y barato.	La verificación se volvería impracticable: cada nodo comprueba millones de hashes por bloque.
Efecto avalancha	Un cambio mínimo altera aproximadamente la mitad de los bits de la salida, de forma impredecible.	Se podrían hacer cambios «pequeños» con hashes parecidos, y la manipulación dejaría de ser evidente.



PREIMAGEN, SEGUNDA PREIMAGEN Y COLISIÓN

PROPIEDAD	QUÉ EXIGE	QUÉ SE ROMPE SI FALLA
Resistencia a preimagen	Dado un hash h , es inviable encontrar ALGUNA entrada m tal que $\text{hash}(m) = h$.	Se podría revertir un hash. Toda la construcción «clave pública $\rightarrow$ dirección» se vendría abajo.
Resistencia a segunda preimagen	Dado un mensaje m_1 , es inviable encontrar otro $m_2 \neq m_1$ con el mismo hash.	Se podría sustituir una transacción por otra distinta conservando el mismo hash: fraude indetectable.
Resistencia a colisiones	Es inviable encontrar CUALQUIER par $m_1 \neq m_2$ con el mismo hash.	Un atacante prepararía de antemano dos documentos con el mismo hash y cambiaría uno por otro después de que el primero fuera aceptado.

[] FICHA DE TÉRMINO 02 · ESTÁN ORDENADAS DE MÁS DIFÍCIL A MÁS FÁCIL

En una preimagen el objetivo está fijo. En una segunda preimagen hay un mensaje dado que igualar. En una colisión el atacante elige LOS DOS mensajes, y esa libertad la hace mucho más barata.

POR QUÉ LAS COLISIONES CUESTAN LA MITAD

 LA PARADOJA DEL CUMPLEAÑOS

En un salón de 23 personas, la probabilidad de que dos compartan cumpleaños supera el 50 %. Sorprende porque uno piensa en «alguien que coincida CONMIGO», cuando lo que cuenta es cualquier par entre todos. Con 23 personas hay 253 pares posibles.

2^n

Coste de encontrar una PREIMAGEN sobre una salida de n bits.

$2^{n/2}$

Coste de encontrar una COLISIÓN. Cada hash nuevo se compara con todos los anteriores.

128

Bits de resistencia real de SHA-256 frente a colisiones, no 256.

Sigue siendo un número inalcanzable, pero explica por qué el tamaño de salida se elige al doble del nivel de seguridad que se busca.



UNA TILDE CAMBIA LA MITAD DE LA SALIDA



134 DE LOS 256 BITS CAMBIARON · 52,3 %

62 de los 64 caracteres hexadecimales son distintos. No hay ninguna relación visible entre las dos salidas — y sin embargo la entrada cambió en un solo carácter.

🗨️ POR QUÉ IMPORTA

El efecto avalancha es lo que hace que toda manipulación sea siempre evidente.



SHA-256 Y KECCAK-256

No hay una sola función hash: hay familias, y cada red eligió la suya. El laboratorio de hoy y todo el código de contratos a partir de la Sesión 6 dependen de esa elección.

FUNCIÓN	ORIGEN	DÓNDE SE USA
SHA-256	Familia SHA-2, publicada por el instituto de estándares estadounidense en 2001.	Bitcoin, en casi todo: identificadores de transacción, árboles de Merkle y prueba de trabajo — habitualmente aplicada dos veces seguidas.
Keccak-256	Algoritmo ganador del concurso público que buscaba el sucesor de SHA-2.	Ethereum, en todo: identificadores, derivación de direcciones, firmas de función en los contratos y el operador keccak256 de Solidity.



POR QUÉ DOS FAMILIAS Y NO UNA

Están construidas sobre principios estructurales distintos, y esa diversidad es deliberada: si apareciera un ataque contra una de las dos familias, la otra no quedaría comprometida por la misma vía. Es el mismo argumento que en la Sesión 1 justificaba tener varias implementaciones del cliente — la monocultura es frágil.



KECCAK-256 NO ES SHA3-256



ESTA DISTINCIÓN CONFUNDE A TODO EL MUNDO AL MENOS UNA VEZ

Keccak ganó el concurso, pero antes de publicarlo como estándar oficial con el nombre SHA-3, el organismo normalizador MODIFICÓ EL RELLENO del algoritmo. Ethereum ya estaba usando la versión original y se quedó con ella.

Resultado: keccak256(x) y sha3_256(x) producen salidas distintas para la misma entrada, aunque el algoritmo de fondo sea el mismo.

Si una biblioteca ofrece «SHA-3», hay que verificar cuál de las dos implementa. Comprobarlo siempre contra un valor conocido antes de confiar en el resultado.



PROBAR SIN ENTREGAR EL CONJUNTO

Planteemos el problema antes de la solución, que es como se entiende.

Un teléfono quiere comprobar que UNA transacción concreta está incluida en un bloque. El bloque contiene miles de transacciones y pesa megabytes. Descargarlo entero para verificar una sola línea es absurdo, y descargar la cadena completa está fuera del alcance de un dispositivo móvil.

¿Se puede demostrar que un elemento pertenece a un conjunto SIN ENTREGAR EL CONJUNTO?

Sí. La estructura que lo hace posible se publicó en 1979 y hoy sostiene tanto la cabecera de un bloque como el direccionamiento por contenido de los sistemas de archivos descentralizados.



ÁRBOL DE MERKLE



FICHA DE TÉRMINO 03 · MERKLE TREE / HASH TREE

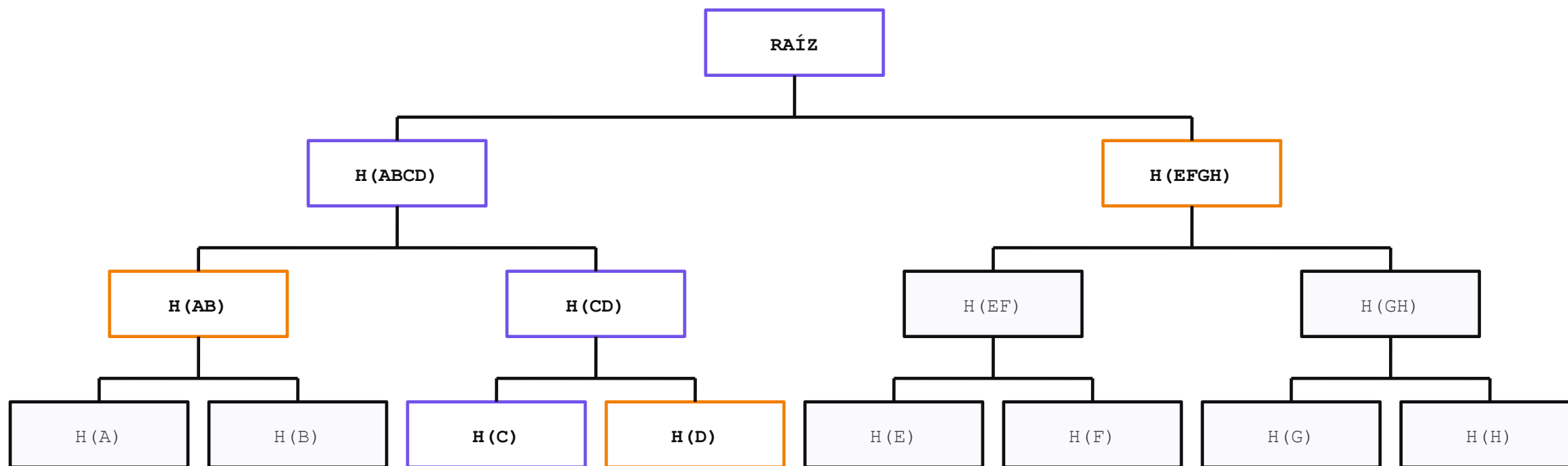
Árbol binario en el que cada hoja es el hash de un dato y cada nodo interno es el hash de la concatenación de sus dos hijos. El nodo superior se llama RAÍZ DE MERKLE y resume todo el conjunto en un solo valor de longitud fija.

La construcción es sencilla: se calcula el hash de cada elemento, se agrupan de dos en dos, se concatenan y se vuelve a hashear; el resultado sube un nivel. Se repite hasta quedar un único valor.

Si el número de elementos de un nivel es impar, la convención más extendida es duplicar el último para completar el par.

Cambiar un solo elemento cambia su hoja, lo que cambia su padre, y así hasta la raíz. LA RAÍZ ES UN COMPROMISO SOBRE EL CONJUNTO ENTERO.

PRUEBA DE INCLUSIÓN: TRES HASHES EN LUGAR DE OCHO HOJAS



PRUEBA DE INCLUSIÓN DE C = H(D) + H(AB) + H(EFGH)

naranja: la prueba · violeta: el camino que se recalcula



PRUEBA DE INCLUSIÓN



FICHA DE TÉRMINO 04 · MERKLE PROOF

Conjunto mínimo de hashes que permite a quien conoce la raíz verificar que un elemento pertenece al conjunto, SIN NECESIDAD DE CONOCER LOS DEMÁS ELEMENTOS.

Quien recibe la prueba parte del dato, calcula su hash, lo combina con el primer hash de la prueba, hashea el resultado, lo combina con el siguiente, y así sucesivamente. Si el valor al que llega coincide con la raíz que ya tenía, el elemento estaba en el conjunto. Si no coincide, no estaba.

Lo notable es lo que NO hace falta: no se necesitan los otros elementos, ni confiar en quien envía la prueba. Una prueba falsa no cuadra con la raíz.



TRAMPA DEL LABORATORIO

La prueba debe guardar si cada hermano estaba a la izquierda o a la derecha. Si se concatena siempre en el mismo orden, la verificación falla la mitad de las veces.



EL CRECIMIENTO LOGARÍTMICO

La prueba tiene tantos hashes como niveles tiene el árbol, es decir $\log_2(n)$. Ese crecimiento tan lento es lo que hace la técnica útil a escala real.

ELEMENTOS EN EL CONJUNTO	HASHES EN LA PRUEBA	TAMAÑO APROXIMADO (32 BYTES CADA UNO)
8	3	96 bytes
1 024	10	320 bytes
1 048 576	20	640 bytes
1 000 000 000	30	960 bytes

Multiplicar el conjunto por mil añade diez hashes a la prueba. Un teléfono verifica la pertenencia a un conjunto de mil millones de elementos con menos de un kilobyte.



DÓNDE VUELVE A APARECER ESTO

EN LA CABECERA DEL BLOQUE

La raíz es uno de los campos de la Fig. 2 de la Sesión 1. Alterar una transacción cambia la raíz, que cambia el hash del bloque, que rompe el enlace del siguiente. Ahora esa cadena es demostrable, no una afirmación.

EN LOS CLIENTES LIGEROS

Es lo que permite que una billetera de teléfono verifique sus transacciones sin descargar la cadena. Retoma la distinción entre nodo completo y ligero de la Sesión 1.

EN LAS LISTAS DE DISTRIBUCIÓN

Se publica una sola raíz en el contrato en lugar de miles de direcciones, y cada beneficiario presenta su prueba. Se usa en la Sesión 10.

EN EL DIRECCIONAMIENTO POR CONTENIDO

La base de cómo IPFS identifica archivos. Se usa en la Sesión 11.



D



MIN 120 – 170 · EN MÁQUINA, EN PAREJAS

EL CÓDIGO, EN VIVO

Recorremos la implementación completa, la ejecutan ustedes, y de ahí sale el trabajo del semestre.

SE ENTREGA RESUELTO, Y SE LEE

El código de esta sesión viene completo. No hay que escribirlo desde cero: hay que ENTENDERLO, ejecutarlo y romperlo. Son setenta líneas y en ellas está toda la Sesión 2.

- | | | |
|----|----------------------------|---|
| 01 | LO RECORREMOS JUNTOS | Cinco láminas, función por función. Cada una responde a una idea que ya vimos en el bloque C. |
| 02 | LO EJECUTAN USTEDES | En sus máquinas, con los ocho valores de referencia. La raíz que obtengan debe coincidir con la de la lámina. |
| 03 | LO ROMPEN | Cambian un carácter de un dato y vuelven a calcular. Ahí se ve el compromiso de la raíz, en su propia pantalla. |
| 04 | LO CONVIERTEN EN ALGO SUYO | El trabajo del semestre parte de aquí: construir una interfaz sobre este código. |

 POR QUÉ ASÍ

Leer código ajeno y entenderlo es una competencia profesional. Escribirlo desde cero, hoy, no lo es todavía.



HASH Y EFECTO AVALANCHA

```
def hash_hex(dato):  
    if isinstance(dato, str):  
        dato = dato.encode("utf-8")  
    return hashlib.sha256(dato).hexdigest()  
  
def bits_distintos(a, b):  
    return bin(int(a, 16) ^ int(b, 16)).count("1")
```

¿POR QUÉ `.encode()`?

SHA-256 hashea BYTES, no texto. Fijar utf-8 explícitamente es lo que hace que dos máquinas obtengan el mismo hash.

¿POR QUÉ XOR?

El XOR pone a 1 exactamente los bits que difieren. Contar los unos da el número de bits distintos — comparar caracteres hexadecimales daría otra cosa: cada carácter son 4 bits.

PROMEDIO SOBRE 100 PARES QUE DIFIEREN EN UN CARÁCTER → $\approx$ 128 DE 256 BITS

Ese número no está pactado en ninguna parte del código: emerge de la función. Es la propiedad del efecto avalancha, medida.



LAS TRES CONVENCIONES, EN TRES LÍNEAS

```
def _combinar(izquierdo, derecho):  
    return hashlib.sha256(bytes.fromhex(izquierdo) + bytes.fromhex(derecho)).hexdigest()
```

Una sola línea, y es donde se pierde la mitad de las parejas. Si en lugar de `bytes.fromhex()` se concatenan las cadenas de texto, el resultado es un hash perfectamente válido — y distinto del de todo el mundo.

1**LAS HOJAS SON EL HASH DEL DATO**

No se mete el dato crudo en el árbol. Se hashea primero.

2**SE CONCATENAN BYTES, NO TEXTO**

`bytes.fromhex()` convierte los 64 caracteres hexadecimales en los 32 bytes que representan.

3**NIVEL IMPAR: SE DUPLICA EL ÚLTIMO**

Es una convención, no una ley matemática. Otras implementaciones eligen distinto — y obtienen otra raíz.



CONSTRUIR EL ÁRBOL: UN BUCLE QUE SUBE

```
def construir_arbol(hojas):  
    nivel = [hash_hex(h) for h in hojas]  
    niveles = []  
    while True:  
        if len(nivel) > 1 and len(nivel) % 2 == 1:  
            nivel = nivel + [nivel[-1]]  
        niveles.append(nivel)  
        if len(nivel) == 1: return niveles  
        nivel = [_combinar(nivel[i], nivel[i+1])  
                 for i in range(0, len(nivel), 2)]
```

8 → 4 → 2 → 1

Cada vuelta del bucle reduce el nivel a la mitad. Por eso la altura del árbol es $\log_2(n)$, y por eso la prueba de inclusión también.

¿POR QUÉ DEVUELVE TODOS LOS NIVELES?

Porque generar_prueba() los necesita: para subir por el árbol hay que saber quién era el hermano en cada escalón.

La línea naranja es la convención 3. Es la única decisión arbitraria de toda la función — y es la que hay que acordar antes de comparar resultados con otra pareja.



GENERAR Y VERIFICAR LA PRUEBA

```
def generar_prueba(hojas, indice):
    niveles = construir_arbol(hojas)
    prueba, idx = [], indice
    for nivel in niveles[:-1]:
        if idx % 2 == 0:
            prueba.append((nivel[idx+1], "der"))
        else:
            prueba.append((nivel[idx-1], "izq"))
        idx = idx // 2
    return prueba
```

```
def verificar_prueba(dato, prueba, raiz):
    actual = hash_hex(dato)
    for hermano, lado in prueba:
        if lado == "izq":
            actual = _combinar(hermano, actual)
        else:
            actual = _combinar(actual, hermano)
    return actual == raiz
```



LAS DOS LÍNEAS NARANJA SON EL MISMO TEMA

A la izquierda, `idx // 2` es «subir un escalón». A la derecha, el orden de la concatenación depende del lado. Si se ignora el lado y se concatena siempre igual, la verificación falla la mitad de las veces — y falla de forma intermitente, que es lo peor que le puede pasar a un error.

Nótese lo que `verificar_prueba` NO recibe: el conjunto de datos. Solo el dato, los hashes hermanos y la raíz. Ahí está toda la utilidad de la estructura.



AHORA CÓRRANLO USTEDES

Quince minutos. En parejas, con el código ya en la máquina.

01

EJECUTAR EL ARCHIVO

`python merkle.py` · o `node merkle.js`

Debe imprimir los dos hashes de referencia, los bits distintos y la raíz de ocho elementos.

02

COMPARAR LA RAÍZ

La raíz de lote-000 ... lote-007 debe empezar por 53c895e4. Si no coincide, revisen las tres convenciones.

03

ROMPERLA

Cambien un solo carácter de un solo dato. Vuelvan a ejecutar. La raíz cambia por completo: el compromiso, en su propia pantalla.

04

PEDIR UNA PRUEBA

`generar_prueba(lotes, 2)` devuelve 3 hashes. Verifiquenla. Después verifiquen un dato que NO está y comprueben que devuelve falso.

RAÍZ DE REFERENCIA · 53c895e4efd5451ab9e313d532203158e907660d0e5d860a5a5cc99ada592fa3



EL VISOR DE ÁRBOLES DE MERKLE

Una página que implementa exactamente estas mismas tres convenciones. Sirve para comprobar el propio código y para ver el árbol completo, que en texto no se aprecia.

EDITAR LOS DATOS

y ver la raíz cambiar al instante, con la anterior tachada al lado.

CLIC EN UNA HOJA

resalta en violeta el camino hasta la raíz y en naranja los hashes que forman la prueba.

VERIFICACIÓN PASO A PASO

cada combinación, con el lado y el valor intermedio, hasta comparar con la raíz.

PROBAR CON UN DATO FALSO

muestra el rechazo. Una verificación que solo acepta lo correcto está a medio escribir.



LO IMPORTANTE

La raíz del visor y la de su código deben ser idénticas. Si no lo son, la diferencia está en una de las tres convenciones.



CONSTRUYAN SU PROPIA INTERFAZ

A partir del código que acaban de ejecutar, construir una interfaz web que lo haga visible. No hay que reimplementar la criptografía: ya está resuelta y probada.

MÍNIMO EXIGIBLE

Un campo para introducir varios datos · la raíz calculada y visible · que la raíz cambie al editar un dato · y que se vea, de alguna forma, la estructura del árbol.

SUMA

Prueba de inclusión al hacer clic en un elemento.

SUMA

Verificación paso a paso, mostrando cada combinación.

SUMA

Rechazo visible de un elemento que no pertenece.

Tecnología libre. Puede ser una sola página con HTML y JavaScript, o el marco de trabajo que prefieran. Lo que se evalúa es que funcione y que se entienda lo que muestra.



ERRORES FRECUENTES · 1 DE 2

SÍNTOMA	CAUSA Y SOLUCIÓN
El hash cambia entre ejecuciones	Se está hasheando un objeto cuya representación varía —un diccionario sin orden fijo, o algo con marca temporal—. Hay que hashear BYTES de una serialización determinista, no la representación de un objeto.
El hash no coincide con el de un compañero	Codificación de texto distinta, o un salto de línea invisible al final de la cadena. Fijar utf-8 explícitamente y comprobar la longitud en bytes antes de hashear.
La raíz no coincide con la de otra pareja	Casi siempre una de tres: se concatenan CADENAS HEXADECIMALES en lugar de BYTES; se hashea el elemento crudo en lugar de su hash al construir las hojas; o se resuelve distinto el nivel impar. Acordar las tres convenciones antes de comparar.



ERRORES FRECUENTES · 2 DE 2

SÍNTOMA	CAUSA Y SOLUCIÓN
La verificación falla la mitad de las veces	No se está respetando el orden izquierda/derecha al concatenar con el hermano. Es el error más común de la parte 3: la prueba debe guardar la posición, no solo el hash.
El promedio de bits distintos da lejos de 128	Se están comparando caracteres hexadecimales en lugar de bits. Cada carácter hexadecimal son 4 bits: hay que convertir a entero y usar XOR, no comparar cadenas posición a posición.
En JavaScript los números salen mal	256 bits no caben en el tipo numérico por defecto. Usar BigInt para las operaciones de bits, o trabajar directamente sobre Buffer byte a byte.
La biblioteca de «SHA-3» da otro resultado	Es la trampa de C.3: unas bibliotecas llaman sha3 a la variante de Ethereum y otras a la del estándar. Comprobar contra un valor conocido antes de confiar.



 MIN 170 — 180 · PLENARIA

CIERRE

Qué llevarse, qué entregar y qué viene en la Sesión 3.

LO QUE HAY QUE LLEVARSE DE ESTA SESIÓN

UNO

Un hash NO es cifrado, ni compresión, ni azar, ni firma. Es una función determinista de un solo sentido.

DOS

Las colisiones EXISTEN NECESARIAMENTE: infinitas entradas, finitas salidas. La seguridad está en que nadie pueda encontrarlas.

TRES

El EFECTO AVALANCHA ya no es una afirmación: cada pareja lo midió y obtuvo alrededor de 128 bits sobre 256.

CUATRO

La RAÍZ DE MERKLE es un compromiso sobre todo el conjunto. Por eso alterar una transacción rompe el bloque, y romper el bloque rompe la cadena.

CINCO

Se puede probar pertenencia con $\text{LOG}_2(N)$ HASHES. Mil millones de elementos caben en una prueba de menos de un kilobyte.

SEIS

La función se elige POR EL PROBLEMA: rápida para verificar cadenas, deliberadamente lenta para guardar contraseñas.

HOY RESOLVIMOS LA MITAD DEL PROBLEMA

Sabemos comprobar que un dato no cambió. Pero un hash lo puede calcular cualquiera, y por lo tanto no dice nada sobre QUIÉN escribió el dato.

En la demostración de apertura hubo un momento en el que el docente pulsó «Confirmar» y la red aceptó la transacción como suya. Nadie mostró un documento de identidad. ¿Cómo se logra eso?

SESIÓN 03 · FIRMAS DIGITALES Y ANATOMÍA DE LA CADENA

Criptografía asimétrica, firma con curva elíptica, y la construcción completa clave privada → clave pública → dirección que hoy quedó dibujada pero no demostrada. Con las firmas disponibles, el laboratorio construye la mini-blockchain con transacciones firmadas de verdad.

GLOSARIO DE LA SESIÓN

Solo los términos que introduce esta sesión. Los de la Sesión 1 no se repiten y siguen siendo exigibles.

Árbol de Merkle	Merkle tree	Árbol binario en el que cada hoja es el hash de un dato y cada nodo interno es el hash de la concatenación de sus dos hijos. Resume un conjunto entero en un único valor.
Colisión		Par de entradas distintas que producen el mismo hash. Existen necesariamente; la seguridad consiste en que sean inencontrables, no inexistentes.
Digest	resumen	Sinónimo de hash: la salida de longitud fija que produce la función. También «huella digital».
Efecto avalancha		Propiedad por la cual un cambio mínimo en la entrada altera aproximadamente la mitad de los bits de la salida, de forma impredecible.
Función hash criptográfica		Función determinista que convierte una entrada de longitud arbitraria en una salida de longitud fija, barata de calcular e inviable de invertir.
Keccak-256		Función hash de 256 bits que usa Ethereum en todo. Corresponde a la versión original del concurso SHA-3 y NO coincide con SHA3-256, que lleva un relleno distinto.



GLOSARIO · CONTINUACIÓN

Paradoja del cumpleaños		Fenómeno por el cual hallar cualquier coincidencia dentro de un conjunto es mucho más barato que hallar una coincidencia con un elemento fijo. Explica que la resistencia a colisiones sea del orden de $2^{n/2}$.
Preimagen		Entrada que produce un hash dado. La resistencia a preimagen exige que, conocido el hash, sea inviable hallar una entrada que lo genere.
Prueba de inclusión	Merkle proof	Conjunto mínimo de hashes hermanos que permite verificar que un elemento pertenece a un conjunto conociendo solo la raíz. Su tamaño crece como $\log_2(n)$.
Raíz de Merkle	Merkle root	Nodo superior del árbol. Compromiso sobre el conjunto completo: cambiar cualquier elemento la cambia. Es uno de los campos de la cabecera de un bloque.
Sal	salt	Valor aleatorio que se añade a una entrada antes de hashearla para que entradas iguales produzcan salidas distintas. Imprescindible con contraseñas; no se usa en una cadena de bloques, cuyos hashes deben ser reproducibles.
Segunda preimagen		Mensaje distinto de uno dado que produce su mismo hash. Más difícil de hallar que una colisión, porque uno de los dos mensajes está fijado.
SHA-256		Función hash de 256 bits de la familia SHA-2. La que usa Bitcoin en identificadores, árboles de Merkle y prueba de trabajo.



FUENTES

FUENTES PRIMARIAS

- Merkle, R. (1979). A Certified Digital Signature. Origen de la estructura.
- Nakamoto, S. (2008). Bitcoin. Sección 7 «Reclaiming Disk Space» y sección 8 «Simplified Payment Verification».
- Especificaciones oficiales de SHA-2 y SHA-3 del instituto de estándares estadounidense.

AMPLIACIÓN RECOMENDADA

- Antonopoulos, A. Mastering Bitcoin. Capítulo sobre estructura del bloque y árboles de Merkle. Libre acceso.
- Narayanan, A. et al. Bitcoin and Cryptocurrency Technologies. Capítulo 1, primitivas criptográficas.
- Documentación de Solidity, entrada del operador keccak256 — anticipa su uso desde la Sesión 6.

PRÓXIMA SESIÓN: FIRMAS DIGITALES

Criptografía asimétrica, ECDSA y la mini-blockchain con transacciones firmadas de verdad.
Entregable de hoy: repositorio con la suite de pruebas pasando, antes de que empiece.